

Kernel Engineer candidate test instructions:

This test exercise consists of 5 problems to solve. To proceed, you must solve them to the best of your ability and submit results for review. These exercises are real-world and are representative of our daily work.

Feel free to use whatever resources you need. Just like in the real world, Google, Stackoverflow, etc. are tools of the trade. However, please make the work your own. Don't copy from other solutions, an AI engine, or ask for outside help. These questions will be used as a basis of discussion in further face to face technical interviews where you will be expected to demonstrate your understanding of what you have submitted.

We do take into consideration experience level and prior background when assessing the submission. For example a recent university graduate would be given much more leeway than a 20 year Linux kernel engineering veteran.

Please tar up all of your responses into a single archive for review and submit to the link provided in the email. Unless prior arrangements have been made, submissions made anywhere besides the link will be ignored.

Q1 Code Review

Code review exercises are important because a significant amount of the Canonical kernel team's workload relies on being able to do quick but thorough code review for patches and bugs submitted. A solid base in code review skills are not only beneficial to kernel maintenance but also kernel software development.

Review the `snippit.c` and create a document (text or pdf, please no specific document types like Word docx) with your answers to submit.

- a. Explain what the code is doing
- b. See if you spot any errors and enumerate them if you find one or more
- c. If you find any errors, explain the issue(s) and how you would resolve them

Q2 Kernel Development Exercise

A smaller but vitally important part of our work is the ability to develop tight clean kernel code.

In this exercise you will:

- Develop a kernel module in C that can be loaded and unloaded into a running Ubuntu 24.04 GA (6.8.x, not HWE or lowlatency) kernel with insmod/rmmod without any errors or warnings, source can be found [here](#)
- The kernel module will create an entry in the proc file system with the ability to read and write binary data by any user
- On each write to the proc file, a copy of the data to be written will be appended to the end of the previously written data, stored in an efficient manner
- Each read from the proc file will return the data written from any provided read offset up to the end of available data, in the sequence it was written
- Assume the proc file will need to be able to handle data interaction from a shell as well as multiple users at the same time
- Properly clean up when the module is unloaded

You are encouraged to demonstrate your knowledge with any additional details not explicitly asked for above, but not to alter the core functionality requested.

We will not respond to requests for clarification on the requirements above. One of the goals of the exercise is to get a feel for how you approach problems and what direction you take to solve them. Don't be afraid of getting a requirement 'wrong', implement what would make sense from a usability perspective given the criteria.

In addition to submitting your actual code and support files, please also include anything you would use to submit this to upstream Linux.

Q3 Kernel Test and Verification

A pillar of the Ubuntu brand is the extensive testing we perform to ensure the most stable and reliable Linux platform available. For this question you will be writing a test suite for the kernel module you developed in Q2 to verify it performs as expected. Given the time constraints, consider test cases that are more impactful before moving on to edge cases.

You may use the programming language of your choice for this exercise.

Q4 Kernel Backporting and Git Usage

This question is to test your skills on Linux kernel backporting and git usage. You will be asked to backport to an Ubuntu kernel a patchset that was submitted to an external mailing list. Please follow the instructions below. For each step below, describe the actions you take and show your work. Even if you do not have sufficient time to complete the entire question, or are unable to finish the entire set of tasks, please turn in the work you have done.

1. The kernel that you need to backport to is the Ubuntu 5.15 Jammy kernel, release tag: Ubuntu-5.15.0-83.92. The kernel can be found at <https://git.launchpad.net/~ubuntu-kernel/ubuntu/+source/linux/+git/jammy> master or master-next branch
2. The patchset that you need to backport is <https://patchwork.kernel.org/project/linux-wireless/cover/20211024152037.332948-1-luca@coelho.fi/>, titled “[00/12] iwlwifi: updates intended for v5.16 2021-10-24 part 2”, which consists of these commits:

```
cbaa6aeedee5 iwlwifi: bump FW API to 67 for AX devices
af84ac579c66 iwlwifi: mvm: extend session protection on association
6905eb1c3b9e iwlwifi: rename CHANNEL_SWITCH_NOA_NOTIF to CHANNEL_SWITCH_START_NOTIF
cf7a7457a362 iwlwifi: mvm: remove session protection on disassoc
a6175a85ba33 iwlwifi: mvm: fix WGDS table print in iwl_mvm_chub_update_mcc()
523de6c872ca iwlwifi: rename GEO_TX_POWER_LIMIT to PER_CHAIN_LIMIT_OFFSET_CMD
4d4cbb9b8e56 iwlwifi: mvm: d3: use internal data representation
9da090cdabcfa iwlwifi: mvm: update RFI TLV
45fe1b6b6c99 iwlwifi: mvm: don't get address of mvm->fwrt just to dereference as a pointer
698b166ed346 iwlwifi: mvm: read 6E enablement flags from DSM and pass to FW
1a5daeade217c iwlwifi: yoyo: support for ROM usniffer
91000fdf8219 iwlwifi: fw: uefi: add missing include guards
```
3. Apply the patchset “[00/12] iwlwifi: updates intended for v5.16 2021-10-24 part 2” on top of the Ubuntu-5.15.0-83.92 tag.
4. You may need to cherry-pick some other commits for resolving any conflicts. The number of required commits to resolve the conflicts should be less than 10.
5. After the backport is done, verify the kernel builds successfully in the “Ubuntu way”, by following the steps listed below:
 1. Install Ubuntu 22.04 release on a machine or in a VM
 2. \$ sudo apt build-dep linux # to prepare build environment
 3. Change directory to the prepared kernel and build using:
\$ fakeroot debian/rules clean binary-genericYou can find more information on how to build a Ubuntu kernel on wiki.ubuntu.com.
6. Export the entire backport as a patch series and write down how you think when backporting commits and submit along with the answers to the other questions.

Q5 Debian Packaging

Ubuntu Linux is based upon the Debian Operating System and therefore utilizes the debian packaging format. In this exercise you will demonstrate your understanding of debian packaging principles and how you would fare in a typical Kernel Team task.

Consider [this](#) Debian DKMS source package from Ubuntu Noble.

Given the following fictional Launchpad bug that could exist at URL <https://launchpad.net/bugs/123456789> (not a real issue, the URL is for example purposes only) with description:

Unset

We need to have the threshold stop with a nice coffee.

Change SMAPI_SET_THRESH_STOP to 0xcafe

Patch the package for the new value to be updated. Make sure your patchset is located in `debian/patches`, as specified by the ``3.0 (quilt)`` in `debian/source/format`.

Provide the content of a `.debdiff` containing the patchset and the new changelog entry.